

# React + Zustand 조립 지도

현재 hospital 프로젝트를 도형으로 이해하는 PDF



React 컴포넌트: 화면 조각



Zustand store: 공용 데이터 창고

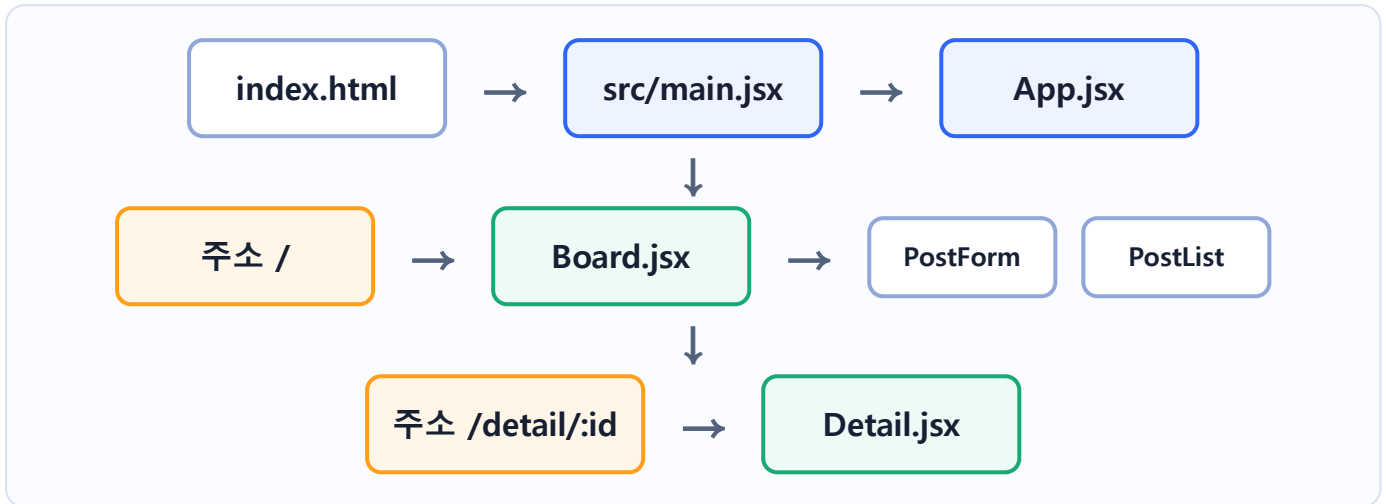


Router: 주소에 따라 화면 고르는 장치

React는 "조립형 화면 만들기"라고 보면 됩니다. 작은 조각을 만들고, 그 조각들을 끼워서 큰 화면을 만듭니다. Zustand는 여러 조각이 같이 쓰는 데이터를 한곳에 모아두는 작은 창고입니다.

## 1. 전체 그림

브라우저가 프로젝트를 읽으면 아래 순서로 화면이 조립됩니다.



**감각** App은 전체 스위치박스입니다. 주소가 `/` 이면 게시판, `/detail/2` 이면 2번 글 상세 화면을 보여줍니다.

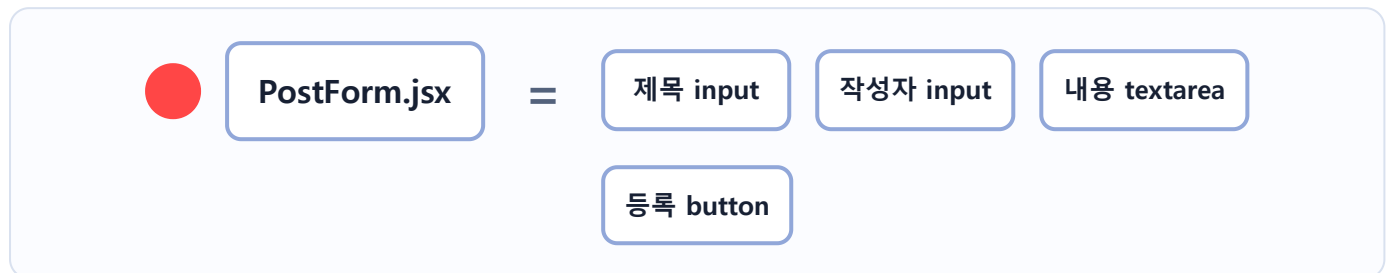
폴더는 이렇게 보면 됩니다

```

hospital └─ index.html 브라우저가 처음 읽는 HTML └─ package.json 실행 명령과 설치된 라이브러리 목록 └─ vite.config.js Vite 설정 └─ src └─ main.jsx React 시작 버튼 └─ App.jsx 주소에 따라 페이지 선택 └─ pages └─ Board.jsx 메인 게시판 페이지 └─ Detail.jsx 상세/수정 페이지 └─ components └─ PostForm.jsx 글쓰기 입력 폼 └─ PostList.jsx 글 목록 테이블 └─ store └─ postStore.js Zustand 데이터 참고
  
```

## 2. React 컴포넌트란?

컴포넌트는 화면 부품입니다. 함수 하나가 HTML처럼 생긴 JSX를 반환하면, React가 그것을 화면에 그립니다.



```
const Board = () => {
  return (
    <main>
      <PostForm />
      <PostList />
    </main>
  )
}
```

위 코드는 “Board라는 큰 판 안에 `PostForm`과 `PostList` 부품을 끼운다”는 뜻입니다.

### Board

큰 게시판 화면입니다. 글 쓰기 폼과 목록을 배치합니다.

### PostForm

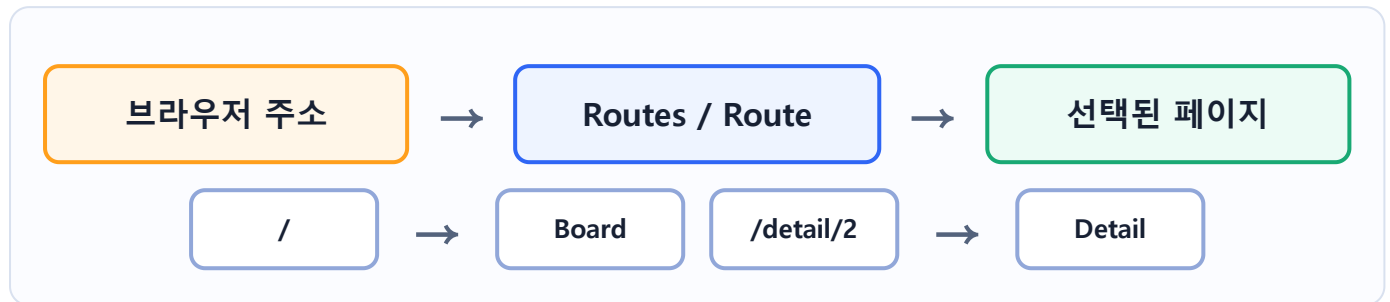
사용자가 새 글 내용을 입력합니다. 등록 버튼을 누르면 store에 글을 넣습니다.

### PostList

store에 있는 글 목록을 표로 보여줍니다. 삭제 버튼도 여기 있습니다.

### 3. Router는 주소 안내원

`react-router-dom`은 현재 주소를 보고 어떤 페이지 컴포넌트를 보여줄지 정합니다.



```

<Routes>
  <Route path="/" element={<Board />} />
  <Route path="/detail/:id" element={<Detail />} />
</Routes>
  
```

`:id`는 빈칸입니다. `/detail/2`로 들어오면 `Detail` 안에서 `useParams()`가 `id = "2"`를 꺼냅니다.

## 4. Zustand는 공용 데이터 창고

예전 방식은 부모 App이 데이터를 들고 자식에게 props로 계속 내려줬습니다. Zustand를 쓰면 필요한 컴포넌트가 store에서 직접 꺼내 씁니다.

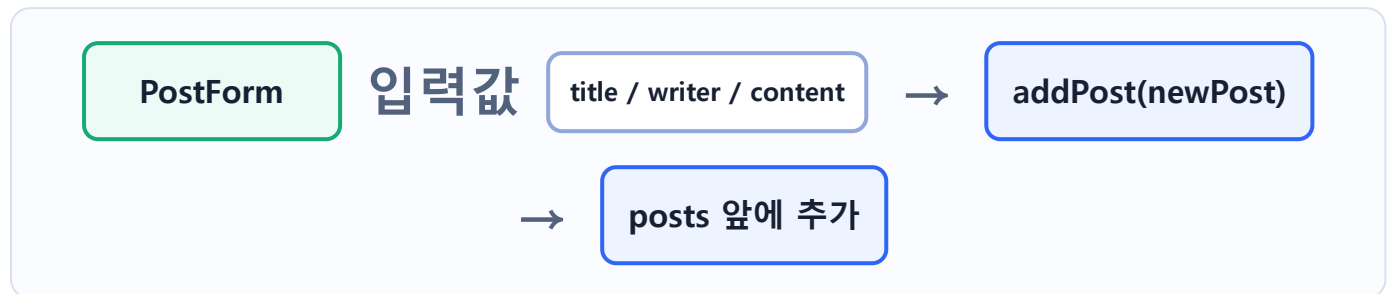


```
const usePostStore = create((set) => ({
  posts: [...],
  addPost: (newPost) => { ... },
  deletePost: (id) => { ... },
  updatePost: (updatePost) => { ... },
}))
```

**감각** `posts` 는 실제 글 목록이고, `addPost`, `deletePost`, `updatePost` 는 창고 안의 데이터를 바꾸는 버튼입니다.

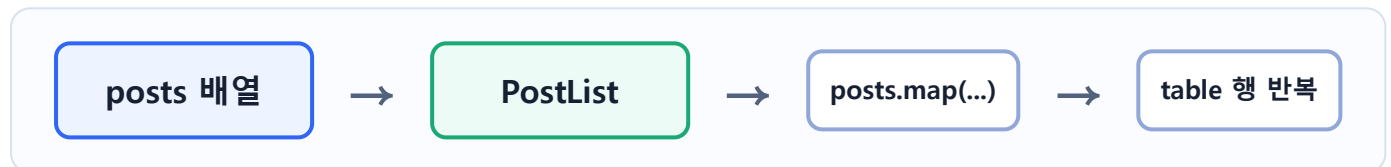
## 5. 현재 데이터 흐름

### Create: 글 등록

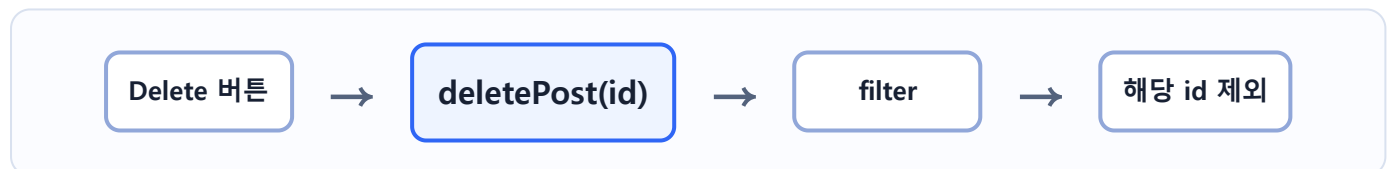


```
const addPost = usePostStore((state) => state.addPost)
addPost({ title, writer, content })
```

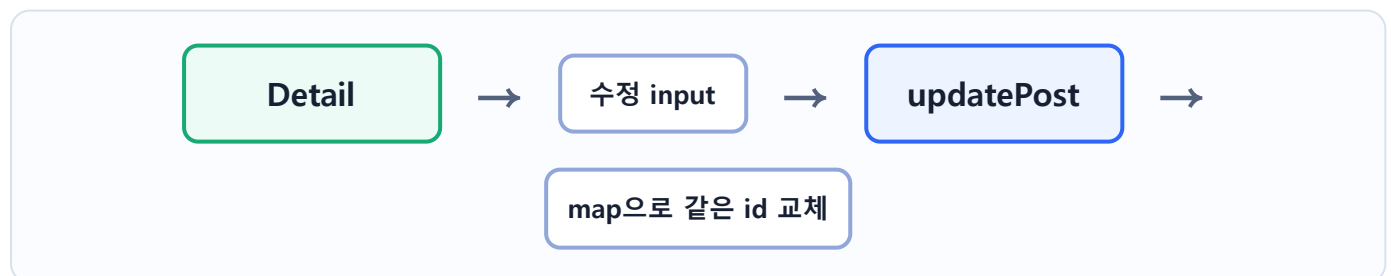
### Read: 목록 보기



### Delete: 삭제



### Update: 수정



## 6. 파일별로 한 문장씩

### main.jsx

React 앱을 `index.html` 의 `#root` 에 끼웁니다. `BrowserRouter` 로 주소 기능도 감쌉니다.

### App.jsx

주소가 `/` 인지 `/detail/:id` 인지 보고 보여줄 페이지를 고릅니다.

### Board.jsx

메인 페이지입니다. 현재는 `PostForm` 과 `PostList` 를 배치합니다.

### PostForm.jsx

입력칸 3개를 `useState` 로 기억하고, 등록 시 `addPost` 를 호출합니다.

### PostList.jsx

Zustand에서 `posts` 를 꺼내 표로 보여주고, 삭제 시 `deletePost` 를 호출합니다.

### Detail.jsx

URL의 id를 읽고 `posts.find` 로 글 하나를 찾습니다. 수정 저장 시 `updatePost` 를 호출합니다.

### postStore.js

글 목록과 CRUD 함수를 담는 공용 창고입니다. 여러 컴포넌트가 여기서 데이터를 꺼냅니다.

### package.json

`npm run dev` , `npm run build` 같은 명령과 React/Zustand 설치 목록이 있습니다.

## 7. 자주 헷갈린 지점

**props 방식과 Zustand 방식을 섞으면 헷갈립니다.**

Zustand를 쓰는 컴포넌트는 보통 `usePostStore` 로 직접 꺼내 쓰고, 중간 컴포넌트가 굳이 props로 전달하지 않아도 됩니다.

### 없는 변수 사용

```
<PostForm addPost={addPost} />
```

위처럼 쓰려면 같은 파일 안에 `addPost` 가 선언되어 있어야 합니다.

### props를 안 받았는데 props 사용

```
const PostList = () => {
  props.posts.length
}
```

`props` 를 쓰려면 `const PostList = (props)` 처럼 받아야 합니다. Zustand로 직접 꺼냈다면 `posts.length` 를 씁니다.

### 배열과 글 하나 구분

```
const posts = state.posts
const post = posts.find(...)
```

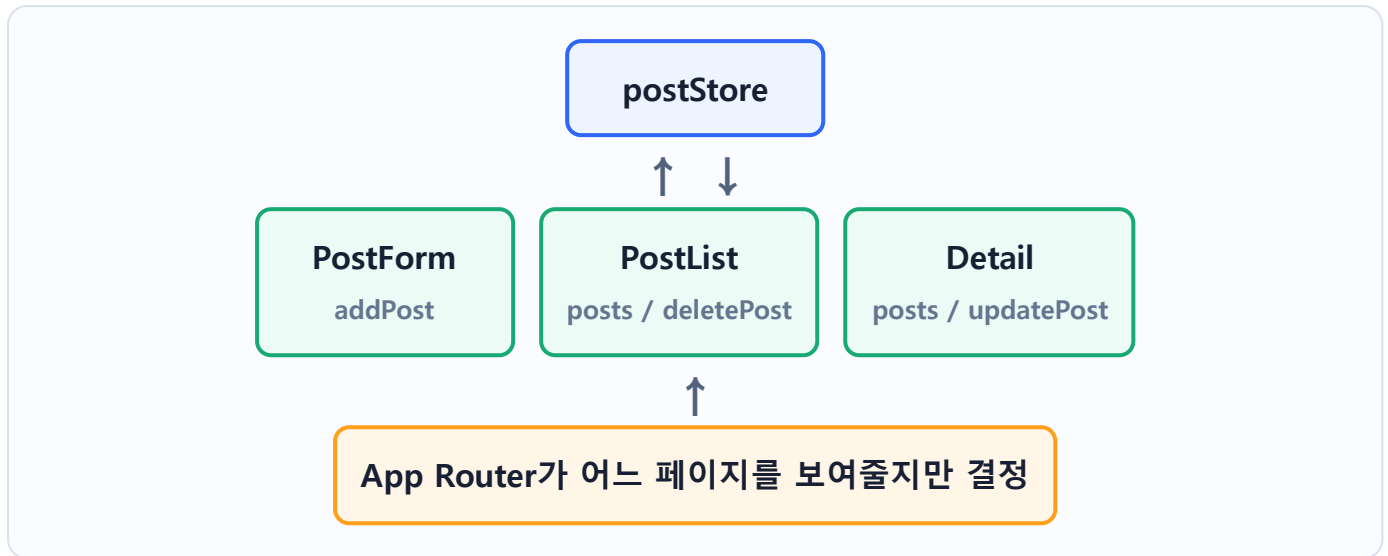
`posts` 는 여러 글이 든 배열, `post` 는 그 중 글 하나입니다.

### 선언 전에 사용

```
const [title] = useState(post.title)
const post = ...
```

이 순서는 안 됩니다. `post` 를 먼저 찾고 나서 `useState` 초기값으로 써야 합니다.

## 8. 머릿속 모델



지금 프로젝트를 이해하는 핵심 문장:

**App**은 길 안내, **Board/Detail**은 화면, **PostForm/PostList**는 작은 부품, **postStore**는 데이터 창고입니다.

### 공부 순서 추천

1. `main.jsx` 에서 앱이 시작되는 모양 보기
2. `App.jsx` 에서 주소와 페이지 연결 보기
3. `Board.jsx` 에서 부품 조립 보기
4. `postStore.js` 에서 데이터와 함수 이름 보기
5. `PostForm.jsx`, `PostList.jsx`, `Detail.jsx` 에서 store를 꺼내 쓰는 부분 찾기